

엔터프라이즈급 일정 관리 애플리케이션 개발 및 클라우드 배포를 위한 종합 엔지니어링 가이드

1. 시스템 아키텍처 및 기술 스택 개요

현대적인 엔터프라이즈 애플리케이션, 특히 구글 캘린더와 같이 복잡한 시간 데이터와 높은 동시성을 처리해야 하는 일정 관리 시스템을 구축하기 위해서는 극도로 최적화된 풀스택 아키텍처가 요구된다. 2026년 중반의 기술 동향을 반영한 본 시스템의 아키텍처는 고성능 처리, 안정성, 그리고 유지보수의 용이성을 달성하기 위해 최신 안정화 버전의 프레임워크와 클라우드 네이티브 배포 전략을 채택한다.

프론트엔드는 최신 버전의 JavaScript(ES2025+)와 React.js를 기반으로 구축된다. 2026년 현재 React 생태계는 성능 최적화가 프론트엔드 엔지니어링의 핵심으로 자리 잡았으며, 초기 React 19 릴리스에서 발견된 React2Shell 취약점 등의 위험을 회피하기 위해 프로덕션 환경에서는 검증된 React 18 LTS(Long-Term Support) 버전이나 완전히 안정화된 최신 버전을 사용하는 것이 강력히 권장된다.¹

백엔드 아키텍처는 Java 21과 Spring Boot 4.0.6을 채택한다. 2025년 말에 처음 도입된 Spring Boot 4와 Spring Framework 7은 Java 17을 최소 요구 사항으로 설정하면서도, 가상 스레드(Virtual Threads)와 같은 최신 JVM 기능을 극대화하기 위해 Java 21 이상의 사용을 강력히 권장하고 있다.² 특히 2026년 4월에 릴리스된 Spring Boot 4.0.6 버전은 탄력적인 클라우드 애플리케이션 지원을 강화함과 동시에, Elasticsearch 및 RabbitMQ 자동 구성 시 발생하는 TLS 호스트 이름 검증 비활성화 취약점(CVE-2026-40970, CVE-2026-40971)과 예측 가능한 임시 디렉토리 소유권 취약점(CVE-2026-40973) 등 65개의 주요 버그와 보안 패치를 포함하고 있어 엔터프라이즈 환경에 필수적이다.³ 데이터 영속성은 시계열 데이터 및 복잡한 간격(Interval) 쿼리에 대해 강력한 인덱싱과 ACID 호환성을 제공하는 PostgreSQL 18을 통해 관리된다.⁴ 특히 PostgreSQL 18은 비동기 I/O(AIO) 하위 시스템을 도입하여 순차 스캔 및 진공(Vacuum) 작업 등에서 비약적인 성능 향상을 제공한다.

배포 토폴로지는 엣지 네트워킹과 컨테이너 오케스트레이션의 이점을 극대화하기 위해 다중 클라우드(Multi-Cloud) 전략을 사용한다. 단일 페이지 애플리케이션(SPA)으로 구성된 프론트엔드는 글로벌 엣지 네트워크를 통해 초저지연 정적 에셋 제공이 가능한 Vercel 플랫폼에 배포된다.⁵ 반면 동적 연산과 데이터베이스 트랜잭션이 발생하는 백엔드 시스템(Spring Boot 애플리케이션 및 PostgreSQL)은 Nixpacks 빌드 시스템을 통해 Java 컨테이너를 자동으로 구성하고 내부 VPC 네트워크를 제공하는 Railway 인프라를 활용하여 구축된다.⁷

2. AI 보조 도구 통합: Claude Code 및 Gemini CLI 환경 구축

소프트웨어 개발 수명 주기(SDLC)에 에이전트형 AI 도구를 통합하는 것은 2026년 엔지니어링의 핵심적인 생산성 향상 전략이다. 그러나 AI 모델이 하위 버전의 기술(예: Spring Boot 2.x/3.x, React의 클래스 컴포넌트)을 환각(Hallucination)하여 레거시 코드를 생성하는 것을 방지하기 위해, 프로젝트 루트에 명시적인 컨텍스트 제어 문서를 구성해야 한다. 본 프로젝트는 터미널 기반의 에이전트 코딩 도구인 Claude Code와 보조 스크립트 생성을 위한 Gemini CLI를 활용한다.⁹

2.1 기본 및 상세 세팅 방법

Claude Code 설치 및 인증: Claude Code는 코드베이스를 직접 읽고, 파일을 편집하며, 명령어를 실행할 수 있는 에이전트 도구이다.⁹ 시스템에 전역으로 설치하기 위해 Node.js 환경에서 다음 명령어를 실행한다.

1. `npm install -g @anthropic-ai/claude-code` 명령어를 통해 CLI를 설치한다.
2. 터미널에서 `claude` 명령어를 입력하여 브라우저 기반의 OAuth 인증을 수행한다.
3. 프로젝트 루트 디렉토리로 이동하여 `claude`를 실행하면 대화형 세션이 시작된다.

Gemini CLI 설치 및 구성:

Gemini CLI는 구조적인 데이터 변환이나 다중 모달(Multi-modal) 분석이 필요할 때 보조적으로 사용된다.

1. `npm install -g @google/gemini-cli`를 통해 설치한다.
2. Google AI Studio에서 발급받은 API 키를 로컬 환경 변수에 등록한다: `export GEMINI_API_KEY="your_api_key_here"`.
3. 터미널에서 `gemini setup`를 실행하여 기본 모델(예: `gemini-1.5-pro`)을 설정한다.

2.2 CLAUDE.md를 통한 정적 컨텍스트 부여

CLAUDE.md는 저장소의 루트에 위치하며, Claude Code가 세션을 시작할 때마다 필수적으로 로드되는 정적 메모리 파일이다. 이 파일은 프로젝트의 목적, 기술 스택, 아키텍처 결정 사항을 AI에게 온보딩(Onboarding)하는 역할을 수행한다.¹¹ 변경이 잦은 정보는 배제하고, AI가 반드시 준수해야 하는 규칙과 최신 공식 문서 링크를 명시해야 한다.¹³

프로젝트 루트의 **CLAUDE.md** 구성 명세서:

프로젝트 구성 - 구글 캘린더 형태의 일정 관리 애플리케이션

핵심 기술 스택 및 버전 제약 조건 (CRITICAL RULES)

본 프로젝트는 2026년 최신 표준을 따르며, AI는 코드 생성 시 반드시 아래의 기술 버전과 제약을 준수해야 한다.

- 프론트엔드: 최신 JavaScript(ES2025+), React 18 LTS 이상.

- 백엔드: Java 21, Spring Boot 4.0.6.
- 데이터베이스: PostgreSQL 18.
- 금지 사항 (**Prohibitions**):
 - Spring Boot 2.x 또는 3.x 패턴을 절대 사용하지 말 것.
 - `javax.*` 패키지 임포트를 절대 금지하며, 반드시 Jakarta EE 11 사양인 `jakarta.*`를 사용할 것.²
 - 레거시 React 클래스 컴포넌트나 안전하지 않은 라이프사이클 메서드를 사용하지 말 것.
 - `uuidv4()` 등 무작위 UUID 생성기를 React 컴포넌트의 `key` 속성으로 사용하는 것을 엄격히 금지함 (성능 저하 방지).¹⁵

공식 문서 참조 링크 (필수 확인)

AI는 코드 생성 전 최신 API 명세를 확인하기 위해 반드시 아래의 문서를 참조한다.

1. **React.js** 공식 문서: <https://react.dev/reference/react>¹⁶
2. **Spring Boot 4.0** 릴리스
 노트: (<https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Release-Notes>)¹⁷
3. **Spring Framework 7** 코어 참조: <https://docs.spring.io/spring-framework/reference/>
4. **Vercel** 라우팅 및 **SPA Rewrite** 설정: <https://vercel.com/docs/routing/rewrites>⁵
5. **Railway Spring Boot** 배포 가이드: <https://docs.railway.com/guides/spring-boot>⁷
6. **Nixpacks Java** 공급자 구성 (**JDK 21**): <https://nixpacks.com/docs/providers/java>⁸

아키텍처 및 개발 패턴

- 백엔드 동시성: Java 21의 가상 스레드를 활용해야 한다
(`spring.threads.virtual.enabled=true`).²
- 데이터 직렬화: Jackson 3를 기본 직렬화 라이브러리로 사용한다.¹⁴
- 상태 관리: 프론트엔드의 일정 데이터 캐싱 및 전역 상태는 Zustand 또는 Redux Toolkit을 통해 관리한다.
- **Null** 안정성: 애플리케이션 코드 내에서 JSpecify 어노테이션(`@Nullable`, `@NonNull`)을 채택하여 Null-safe API를 구성한다.¹⁴

2.3 SKILLS.md를 통한 동적 에이전트 스킬 구현

정적 컨텍스트인 CLAUDE.md와 달리, SKILLS.md는 AI가 특정 작업을 수행할 때만 동적으로 로드되는 재사용 가능한 전문 지식 패키지이다.¹² 이를 통해 토큰 컨텍스트 윈도우의 압박을 줄이면서도 복잡한 프론트엔드 컴포넌트 생성이나 데이터베이스 마이그레이션과 같은 반복 작업을 자동화할 수 있다.¹⁸

일정 관리 애플리케이션을 위해 프로젝트 내 `.claude/skills/react-calendar-component/SKILL.md` 경로에 다음과 같은 스킬을 정의한다.

SKILL.md 구성 명세서:

YAML

```
---
name: generate-calendar-component
description: 구글 캘린더 스타일의 일정 관리 앱을 위한 고성능 React 컴포넌트를 생성하거나 리팩토링할 때
트리거된다.
---
# React 일정 컴포넌트 엔지니어링 가이드

AI 도구가 캘린더 UI 컴포넌트를 설계할 때 다음 원칙을 반드시 적용한다:
1. **렌더링 최적화**: 캘린더의 월간/주간 뷰는 매우 많은 DOM 노드를 생성한다. 따라서 `React.memo`,
`useMemo`, `useCallback`을 적극 활용하여 불필요한 하위 컴포넌트의 리렌더링을 방지해야 한다.
2. **가상화(Virtualization)**: 이벤트 목록이 길어지거나 타임라인이 복잡한 경우 `react-window`와 같은
가상화 기법을 적용하여 뷰포트 내의 요소만 렌더링하도록 강제한다.
3. **웹 접근성(A11y)**: 시맨틱 HTML을 기반으로 ARIA 역할(Roles)과 키보드 내비게이션(방향키를 통한 날짜
이동) 기능을 필수적으로 구현한다.
4. **스타일링**: CSS Modules 또는 Tailwind CSS를 사용하여 컴포넌트의 스타일 충돌을 방지하고 유연한
테마(Light/Dark) 시스템을 지원한다.
```

3. 프론트엔드 아키텍처 및 컴포넌트 구조도

구글 캘린더와 동일한 사용자 경험(UX)을 제공하기 위해서는 복잡한 달력 렌더링 로직과 이벤트의 드래그 앤 드롭(Drag and Drop), 다중 타임존 처리 등 매우 고도화된 프론트엔드 아키텍처가 필요하다. 단일 페이지 애플리케이션(SPA) 내에서 상태 변화에 따른 즉각적인 UI 피드백을 제공하기 위해 컴포넌트 트리를 전략적으로 분리한다.

3.1 컴포넌트 계층 구조 및 역할

전체 애플리케이션은 상태를 주입하는 컨테이너(Container) 컴포넌트와 화면을 렌더링하는 프레젠테이션(Presentational) 컴포넌트로 명확하게 분리된다.

- **AppRoot**: 애플리케이션의 최상위 진입점이다. React Router의 BrowserRouter, 인증 컨텍스트(Auth Provider), 테마 제공자(Theme Provider), 그리고 전역 상태 관리 저장소(예: Zustand Store)를 감싸는 역할을 한다.
- **LayoutContainer**: 좌측의 사이드바와 우측의 메인 달력 영역을 분할하는 구조적 컴포넌트이다.
- **Sidebar** 컴포넌트 그룹:
 - **MiniCalendar**: 빠른 날짜 이동을 위한 축소판 월간 달력이다. 선택된 날짜 상태를 전역 저장소에 업데이트한다.
 - **CalendarFilters**: '개인', '업무', '휴일' 등 일정의 카테고리를 필터링할 수 있는 체크박스

리스트 컴포넌트이다.

- **MainCalendarGrid** 컴포넌트 그룹: 핵심적인 렌더링 영역으로, 사용자가 선택한 뷰 타입(월/주/일)에 따라 동적으로 하위 컴포넌트를 교체한다.
 - **MonthView**: 6행 7열의 그리드를 렌더링한다. 각 날짜 셀(Cell)은 해당 일자의 이벤트 배열을 매핑하여 렌더링한다.
 - **WeekView** 및 **DayView**: 세로축을 24시간 단위(보통 15분 단위의 격자)로 나누어 일정을 시각적인 시간 블록으로 렌더링한다. 절대 위치(Absolute Positioning)를 사용하여 이벤트의 겹침(Overlap)을 계산하고 렌더링해야 한다.
- **EventInteraction** 컴포넌트 그룹:
 - **EventModal**: 일정 등록, 조회, 수정, 삭제(CRUD) 폼을 제공하는 플로팅 모달 창이다. 사용자 입력을 받아 API 호출을 트리거한다.
 - **DraggableEvent**: 캘린더 그리드 위에 렌더링되는 개별 일정 블록으로, HTML5 Drag & Drop API 또는 서드파티 라이브러리(예: **dnd-kit**)를 활용하여 일정을 다른 날짜나 시간으로 이동시키는 기능을 제공한다.

3.2 렌더링 성능 최적화 전략

월간 뷰(Month View)에서 다수의 일정이 존재하는 경우, 상위 컴포넌트의 상태가 변경될 때마다 하위의 모든 날짜 셀과 일정 컴포넌트가 다시 렌더링되는 성능 병목 현상이 발생할 수 있다.

2026년 프론트엔드 환경에서 이러한 성능 저하는 치명적인 아키텍처 실패로 간주된다.¹⁵

이를 해결하기 위해, 각 날짜 셀(Day Cell) 컴포넌트는 **React.memo**로 감싸져야 하며, 상위 컴포넌트로부터 전달받는 이벤트 데이터 필터링 로직은 반드시 **useMemo**를 통해 캐싱되어야 한다. 또한, 캘린더 내에서 컴포넌트의 리스트를 매핑할 때 고유 식별자로 **Math.random()**이나 렌더링 시마다 새로 생성되는 **uuidv4()**를 사용하는 것은 절대 금지된다. 식별자가 렌더링마다 변경되면 **React**의 재조정(**Reconciliation**) 알고리즘이 기존 **DOM** 트리를 완전히 파괴하고 새로 생성하기 때문에 심각한 성능 낭비를 초래한다.¹⁵ 따라서 데이터베이스에서 반환된 안정적인 기본 키(**Primary Key**, 예: 데이터베이스에서 생성된 **UUID**)를 **key** 속성으로 사용해야 한다.

4. 백엔드 아키텍처 및 데이터베이스 설계

백엔드는 대규모 동시 요청을 안정적으로 처리하기 위해 **Java 21**과 **Spring Boot 4.0.6**을 활용하여 마이크로서비스 지향적인 모놀리스(**Modular Monolith**) 구조로 설계된다. **Spring Boot 4**는 이전 버전과 달리 **javax.*** 네임스페이스를 완전히 버리고 **Jakarta EE 11** 사양을 채택하였으며, 내장형 서버로서 **Undertow**의 지원을 중단하고 최적화된 **Tomcat**을 기본으로 사용한다.²

4.1 Java 21 가상 스레드(Virtual Threads)의 적용

일정 관리 앱은 본질적으로 다수의 사용자가 동시에 자신의 일정을 조회하거나 수정하는 **I/O 바운드(I/O-Bound)** 작업이 주를 이룬다. 기존의 **OS 스레드 풀** 모델에서는 동시 접속자가 급증할 경우 스레드 고갈 현상이 발생할 수 있다. 본 아키텍처에서는 **application.yml** 파일 내에 **spring.threads.virtual.enabled: true** 속성을 명시적으로 선언함으로써 **Java 21**의 가상 스레드 기능을 활성화한다.² 가상 스레드는 **OS 스레드**와 **1:1**로 매핑되지 않고 **JVM** 내부에서 초경량으로 관리되므로, 블로킹 **I/O** 작업 중에도 스레드를 반환하여 수만 개의 동시 연결을 적은 메모리

오버헤드로 처리할 수 있다.

4.2 도메인 모델 및 null 안정성 (Null-Safety) 적용

Spring Boot 4 생태계의 모든 프로젝트는 JSpecify null 안정성 어노테이션을 채택하였다.¹⁴ 이는 컴파일 단계에서 `NullPointerException`의 발생 가능성을 사전에 차단하는 강력한 도구이다.

엔티티 설계 시 다음과 같은 패턴을 적용한다.

Java

```
import jakarta.persistence.*;
import org.jspecify.annotations.NonNull;
import org.jspecify.annotations.Nullable;
import java.time.OffsetDateTime;
import java.util.UUID;

@Entity
@Table(name = "calendar_events")
public class CalendarEvent {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @NonNull
    @Column(nullable = false, length = 255)
    private String title;

    @Nullable
    @Column(columnDefinition = "TEXT")
    private String description;

    @NonNull
    @Column(name = "start_time", nullable = false)
    private OffsetDateTime startTime;

    @NonNull
    @Column(name = "end_time", nullable = false)
    private OffsetDateTime endTime;
```

```
// 연관관계 및 기본 생성자 생략
}
```

일정의 시작 시간(startTime)과 종료 시간(endTime)은 반드시 시간대(Timezone) 정보를 포함하는 `OffsetDateTime` 혹은 `ZonedDateTime`으로 선언되어야 한다. 이를 통해 글로벌 사용자가 각자의 현지 시간대에서 일정을 생성하더라도, 데이터베이스에는 UTC를 기준으로 명확하게 저장 및 변환될 수 있다.

4.3 PostgreSQL 데이터베이스 설계 및 스키마

Railway 플랫폼에서 호스팅되는 PostgreSQL 18은 다음과 같은 관계형 데이터베이스 스키마를 갖는다.⁴ 일정 조회 성능을 극대화하기 위해 적절한 인덱스 전략이 수반되어야 한다.

테이블명	컬럼명	데이터 타입	제약 조건	설명
users	id	VARCHAR(255)	PRIMARY KEY	사용자의 고유 식별자
users	email	VARCHAR(255)	UNIQUE, NOT NULL	이메일 주소
users	timezone	VARCHAR(50)	NOT NULL	사용자의 기본 시간대 (예: Asia/Seoul)
calendar_events	id	UUID	PRIMARY KEY	일정의 고유 식별자
calendar_events	user_id	VARCHAR(255)	FOREIGN KEY, NOT NULL	일정을 소유한 사용자의 식별자
calendar_events	title	VARCHAR(255)	NOT NULL	일정 제목

calendar_events	description	TEXT	NULL	일정에 대한 상세 설명
calendar_events	start_time	TIMESTAMPTZ	NOT NULL	시간대가 포함된 일정 시작 시간
calendar_events	end_time	TIMESTAMPTZ	NOT NULL	시간대가 포함된 일정 종료 시간

인덱싱 전략: 프론트엔드가 월간 뷰를 렌더링할 때 "특정 사용자의 특정 기간(예: 5월 1일 ~ 5월 31일) 내의 모든 일정"을 조회하는 쿼리가 빈번하게 발생한다. 따라서 `calendar_events` 테이블에 `user_id`, `start_time`, `end_time`을 결합한 복합 인덱스(Composite Index)를 생성하여 Full Table Scan을 방지하고 조회 성능을 최적화해야 한다.

5. RESTful API 명세서

Spring Boot 기반의 컨트롤러(Controller)는 클라이언트와 데이터를 주고받기 위해 RESTful 원칙을 준수하는 엔드포인트를 제공한다. 데이터 전송 객체(DTO)는 Java의 `record` 클래스를 활용하여 불변 객체로 설계하며, 직렬화는 Spring Boot 4의 기본 설정인 Jackson 3를 통해 수행된다.¹⁴ 모든 요청과 응답은 JSON 형식을 따른다.

5.1 API 엔드포인트 명세 테이블

HTTP 메서드	엔드포인트 URI	목적 및 기능 설명	요청 페이로드 (Payload)	성공 응답 코드 및 데이터
POST	/api/v1/events	새로운 일정을 데이터베이스에 등록한다.	EventCreateRequestDTO	201 Created / 생성된 일정 객체
GET	/api/v1/events	특정 기간 내의 일정을 조회한다. (월간/주간 뷰)	Query 파라미터: start, end	200 OK / 일정 객체의 배열(List)

		렌더링용)		
GET	/api/v1/events/{id}	특정 일정의 상세 정보를 단건 조회한다.	없음	200 OK / 단일 일정 객체
PUT	/api/v1/events/{id}	기존 일정의 제목, 시간, 내용 등을 수정한다.	EventUpdateRequestDTO	200 OK / 수정된 일정 객체
DELETE	/api/v1/events/{id}	특정 일정을 데이터베이스에서 삭제한다.	없음	204 No Content / 본문 없음

5.2 DTO 및 예외 처리(Exception Handling)

Jakarta Bean Validation 3.1을 사용하여 API 요청 데이터를 엄격하게 검증한다.²

Java

```
public record EventCreateRequestDTO(
    @NotBlank(message = "일정 제목은 필수입니다.")
    String title,

    String description,

    @NotNull(message = "시작 시간은 필수입니다.")
    OffsetDateTime startTime,

    @NotNull(message = "종료 시간은 필수입니다.")
    OffsetDateTime endTime
) {}
```

종료 시간이 시작 시간보다 앞설 수 없는 등의 비즈니스 로직 검증은 서비스(Service) 계층에서

수행된다. 검증이 실패하거나 예외가 발생할 경우, 전역 예외 처리기(`@RestControllerAdvice`)가 이를 포착하여 RFC 7807 표준인 Problem Details 형식으로 클라이언트에 일관된 에러 응답을 반환해야 한다.

6. 보안 및 환경 변수(.env) 관리 전략

데이터베이스 자격 증명, API 키, 서버 포트와 같은 민감한 정보는 절대 소스 코드(GitHub 저장소 등)에 평문으로 하드코딩되어서는 안 된다. 시스템은 12-Factor App 방법론에 따라 구성과 코드를 엄격히 분리하며, 모든 민감 정보는 환경 변수를 통해 주입된다.

6.1 백엔드(Spring Boot) 환경 변수 바인딩

Spring Boot의 `application.yml` 파일은 `${환경변수명:기본값}` 문법을 사용하여 실행 환경(OS)의 환경 변수를 동적으로 읽어온다.²²

YAML

```
spring:
  datasource:
    url: jdbc:postgresql://${PGHOST:localhost}:${PGPORT:5432}/${PGDATABASE:calendar_db}
    username: ${PGUSER:postgres}
    password: ${PGPASSWORD:secret}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
server:
  port: ${PORT:8080}
```

로컬 개발 환경에서는 `.env` 파일이나 IDE의 Run Configuration을 통해 환경 변수를 주입하여 실행하며, 프로덕션 인프라인 Railway에서는 인프라 자체가 동적으로 이 변수들을 컨테이너 시작 시 주입한다.⁴

6.2 프론트엔드(React) 환경 변수 격리

React 프론트엔드는 빌드 도구(예: Vite 또는 Webpack)에 따라 특정한 접두사를 가진 환경 변수만을 클라이언트 측 코드에 노출시킨다 (예: Vite의 경우 `VITE_`, Create React App의 경우 `REACT_APP_`).

- 로컬 환경 **.env.local**: `VITE_API_BASE_URL=http://localhost:8080/api/v1`
- 프로덕션 환경 (**Vercel** 설정): `VITE_API_BASE_URL=https://api.mycalendar.com/api/v1`

이러한 분리를 통해, 프론트엔드 코드는 자신이 로컬 서버와 통신하는지 프로덕션 서버와

통신하는지 알 필요 없이 오직 `import.meta.env.VITE_API_BASE_URL`를 참조하여 API를 호출하게 된다. 주의할 점은 프론트엔드 `.env` 파일에는 백엔드 데이터베이스 비밀번호와 같은 시스템 내부의 비밀 키를 절대 포함해서는 안 된다는 것이다.

7. 클라우드 인프라 배포 가이드

최종 구축된 일정 관리 앱은 Vercel과 Railway를 통해 프로덕션 환경에 배포된다. 각 플랫폼의 특성을 이해하고 라우팅 및 런타임 설정을 올바르게 구성해야 무중단 서비스를 제공할 수 있다.

7.1 프론트엔드 배포: Vercel SPA 라우팅 구성

Vercel은 GitHub 저장소와의 연동을 통해 코드가 main 브랜치에 푸시될 때마다 자동으로 빌드(`npm run build`)를 수행하고 배포한다. 하지만 React 기반의 SPA(Single Page Application)를 정적 호스팅 서비스인 Vercel에 배포할 때 라우팅 문제가 발생할 수 있다.

사용자가 `https://my-calendar-app.com/events/123`와 같은 특정 URL로 직접 접속하거나 새로고침을 할 경우, Vercel 서버는 해당 경로와 일치하는 물리적인 `index.html` 파일을 찾지 못해 404 Not Found 에러를 반환한다. 모든 라우팅 로직(React Router)은 클라이언트 측 브라우저에서 처리되어야 하므로, Vercel 서버로 들어오는 모든 요청을 단일 진입점인 루트의 `index.html`로 재작성(Rewrite)해야 한다.⁵

이를 해결하기 위해 프론트엔드 프로젝트의 루트 디렉토리에 반드시 `vercel.json` 파일을 작성하여 포함시킨다.⁵

JSON

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

이 설정을 통해 외부 API로 향하지 않는 애플리케이션 내의 모든 동일 애플리케이션(Same-application) 라우팅 요청이 정상적으로 React 애플리케이션으로 전달된다.⁵

7.2 백엔드 및 DB 배포: Railway 인프라 구성

Railway는 코드를 분석하여 자동으로 컨테이너를 빌드하고 배포하는 플랫폼이다.²² 이 플랫폼에서 Spring Boot와 PostgreSQL 18을 안정적으로 통합하기 위해 다음 단계를 거친다.

1단계: PostgreSQL 서비스 프로비저닝 및 변수 생성 Railway 대시보드에서 'New Project'를 생성하고 PostgreSQL 18 데이터베이스 서비스를 추가한다. 인프라가 구성되면 Railway는 PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD와 같은 연결 참조 변수(Reference Variables)를 자동으로 생성한다.⁴ 중요한 점은, Spring Boot 애플리케이션이 이 데이터베이스에 연결할 때 외부 프록시 URL이 아닌 내부 네트워크 URL(예: mysql.railway.internal 또는 PostgreSQL의 내부 도메인)을 사용해야 네트워크 지연과 에러(UnknownHostException 등)를 방지할 수 있다는 점이다.²⁴

2단계: Java 21 버전을 위한 Nixpacks 설정 Railway는 애플리케이션 빌드를 위해 Nixpacks라는 도구를 사용한다.²⁷ 그러나 Nixpacks의 Java 공급자는 기본적으로 JDK 17을 사용한다.⁸ 본 프로젝트는 가상 스레드 활용을 위해 Java 21이 필수적이므로, 배포 전 Railway의 Spring Boot 서비스 환경 변수(Variables) 탭에서 빌드 버전을 명시적으로 재정의해야 한다.²⁸

- 환경 변수 설정: NIXPACKS_JDK_VERSION=21⁸

만약 이 변수를 설정하지 않으면 빌드 단계에서 Fatal error compiling: error: release version 21 not supported 에러가 발생하며 배포가 실패하게 된다.²⁸

3단계: Shared Variables를 통한 환경 변수 매핑 Spring Boot 서비스와 PostgreSQL 서비스 간에 환경 변수가 올바르게 공유되어야 한다. Railway의 공유 변수 기능을 사용하거나, Spring Boot 서비스의 설정에서 데이터베이스가 제공하는 PG* 변수들을 application.yml이 기대하는 형식에 맞추어 주입해야 한다.⁴ 변수가 정상적으로 바인딩되지 않으면 애플리케이션 시작 시 Failed to initialize JPA EntityManagerFactory 에러가 발생하며 서버가 종료된다.⁴

4단계: 퍼블릭 도메인 노출 배포가 완료되면 Railway 서비스 설정의 'Networking' 섹션에서 'Generate Domain'을 클릭하여 외부에서 API에 접근할 수 있는 퍼블릭 URL을 생성한다.⁷ 이 도메인을 프론트엔드의 VITE_API_BASE_URL 환경 변수에 등록한다.

8. 엔지니어링 표준 및 코딩 컨벤션

다수의 개발자가 참여하거나 AI 도구와 협업하는 프로젝트에서는 일관된 코드 품질과 추적 가능성을 유지하기 위해 엄격한 엔지니어링 표준이 요구된다.

8.1 프론트엔드 및 백엔드 코딩 컨벤션

프론트엔드 (React/JavaScript):

- 린팅 및 포매팅: ESLint와 Prettier를 프로젝트에 구성하고, 저장 시 자동 포매팅(Format on Save)이 되도록 IDE를 설정한다. 특히 eslint-plugin-react-hooks를 통해 React 훅스의 의존성 배열 누락을 엄격하게 감시한다.
- 컴포넌트 선언: 모든 컴포넌트는 화살표 함수(Arrow Functions) 형태로 작성하며, Props의 타입 검증은 TypeScript를 사용하거나 불가피한 경우 PropTypes를 통해 명시한다.
- 경로 별칭 (Path Aliases): ../..../components/Button과 같은 지저분한 상대 경로 임포트를

방지하기 위해 빌드 도구 설정(예: vite.config.js 또는 tsconfig.json)에서 @/components, @/api와 같은 절대 경로 별칭을 구성하여 사용한다.

백엔드 (Java/Spring Boot):

- **Google Java Style Guide** 준수: 들여쓰기, 중괄호 위치, 네이밍 규칙 등은 구글 자바 스타일 가이드를 따른다. 빌드 도구(Maven/Gradle)에 Spotless 또는 Checkstyle 플러그인을 통합하여 CI 과정에서 코드 스타일을 강제로 검증한다.
- 계층 간 역할 분리(**Separation of Concerns**): 컨트롤러(Controller) 계층에는 어떠한 비즈니스 로직도 포함되어서는 안 된다. 컨트롤러는 HTTP 요청을 받아 DTO를 매핑하고, 비즈니스 로직은 서비스(Service) 계층으로 위임하며, 서비스 계층에서 트랜잭션 관리(@Transactional)를 전담한다.

8.2 깃허브 커밋 컨벤션 (Conventional Commits)

코드의 변경 이력을 명확하게 추적하고, 향후 자동화된 릴리스 노트 및 버전 관리(Semantic Versioning)를 가능하게 하기 위해 모든 커밋 메시지는 **Conventional Commits** 사양을 엄격하게 따른다.

커밋 메시지 형식: <타입>(<스cope>): <제목>

- 타입(**Type**) 정의:
 - **feat**: 새로운 기능 추가 (예: feat(calendar): 월간 뷰 이벤트 드래그 앤 드롭 기능 추가)
 - **fix**: 버그 수정 (예: fix(api): 시간대 변환 시 발생하는 오프셋 계산 오류 수정)
 - **docs**: 문서 수정 (예: docs(readme): Railway 배포 관련 환경 변수 설정 가이드 추가)
 - **style**: 코드 포매팅 변경, 세미콜론 누락 수정 (비즈니스 로직 변경 없음)
 - **refactor**: 코드 리팩토링 (예: refactor(db): 무작위 UUID 생성을 데이터베이스 기본 전략으로 변경)
 - **test**: 테스트 코드 추가 및 수정
 - **chore**: 빌드 스크립트 수정, 패키지 매니저 환경설정 변경
- 스코프(**Scope**): 변경이 발생한 모듈이나 컴포넌트를 괄호 안에 명시하여 변경 범위를 국소화한다 (선택 사항이나 권장됨).
- 제목(**Subject**): 명령문 형태의 간결한 한국어(또는 영어)로 변경 사항을 설명하며, 마침표를 찍지 않는다.

결론적으로, 본 가이드에서 명시한 아키텍처적 결단—Java 21 가상 스레드 기반의 Spring Boot 4.0.6 백엔드, React 18 LTS 프론트엔드, 그리고 다중 클라우드 배포 파이프라인—은 엔터프라이즈 수준의 일정 관리 시스템이 요구하는 고가용성 및 성능 요건을 충족시킨다. 더불어 Claude Code와 Gemini CLI 같은 AI 보조 도구의 강력한 역량을 안전하게 제어하기 위한 CLAUDE.md 및 SKILLS.md 기반의 컨텍스트 샌드박스 전략은 현대 개발 파이프라인의 핵심적인 생산성 레버리지로 작용할 것이다.

참고 자료

1. React Versions Explained: Why LTS Matters and Which Versions Are Safe - Medium, 5월 17, 2026에 액세스,

- <https://medium.com/@sindhujad6/react-versions-explained-why-lts-matters-and-which-versions-are-safe-c53a984e62a3>
2. Spring Boot 4 & Spring Framework 7 – What's New - Baeldung, 5월 17, 2026에 액세스, <https://www.baeldung.com/spring-boot-4-spring-framework-7>
 3. Spring Boot 4.0.6 available now, 5월 17, 2026에 액세스, <https://spring.io/blog/2026/04/23/spring-boot-4-0-6-available-now>
 4. Spring Boot PostgreSQL Environment Variable Injection Failure on Railway Deployment, 5월 17, 2026에 액세스, <https://station.railway.com/questions/spring-boot-postgre-sql-environment-varia-da18bc2e>
 5. Rewrites on Vercel, 5월 17, 2026에 액세스, <https://vercel.com/docs/routing/rewrites>
 6. React Router on Vercel, 5월 17, 2026에 액세스, <https://vercel.com/docs/frameworks/frontend/react-router>
 7. Deploy a Spring Boot App | Railway Guides, 5월 17, 2026에 액세스, <https://docs.railway.com/guides/spring-boot>
 8. Java | Nixpacks, 5월 17, 2026에 액세스, <https://nixpacks.com/docs/providers/java>
 9. Claude Code overview - Claude Code Docs, 5월 17, 2026에 액세스, <https://code.claude.com/docs/en/overview>
 10. Using spec-driven development with Claude Code | by Heeki Park - Medium, 5월 17, 2026에 액세스, <https://heeki.medium.com/using-spec-driven-development-with-claude-code-4a1ebe5d9f29>
 11. Writing a good CLAUDE.md | HumanLayer Blog, 5월 17, 2026에 액세스, <https://www.humanlayer.dev/blog/writing-a-good-claude-md>
 12. What's the difference between skills.md, agents.md, and claude.md? : r/LLM - Reddit, 5월 17, 2026에 액세스, https://www.reddit.com/r/LLM/comments/1qtlizp/whats_the_difference_between_skillsmd_agentsmd/
 13. Best practices for Claude Code - Claude Code Docs, 5월 17, 2026에 액세스, <https://code.claude.com/docs/en/best-practices>
 14. Spring Boot 4.0 Release Highlights, 5월 17, 2026에 액세스, <https://spring.io/projects/release-highlights/>
 15. React 18 changed the game. And in 2026, it's expected knowledge. - DEV Community, 5월 17, 2026에 액세스, <https://dev.to/er-raj-aryan/react-18-changed-the-game-and-in-2026-its-expected-knowledge-3887>
 16. React Versions, 5월 17, 2026에 액세스, <https://react.dev/versions>
 17. Spring Boot 4.0 Release Notes - GitHub, 5월 17, 2026에 액세스, <https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Release-Notes>
 18. SKILLS.md: The New Primitive for Building AI Agents | by Michiel Horstman - Medium, 5월 17, 2026에 액세스, <https://michiellh.medium.com/skills-md-the-new-primitive-for-building-ai-agents-e3d3f024b6c5>

19. claude-code-interview/claude-code-Skills.md at main · huifer/claude-code-interview - GitHub, 5월 17, 2026에 액세스, <https://github.com/huifer/claude-code-interview/blob/main/claude-code-Skills.md>
20. Spring Boot 4.0.0 available now, 5월 17, 2026에 액세스, <https://spring.io/blog/2025/11/20/spring-boot-4-0-0-available-now/>
21. Spring Boot 4.0 Migration Guide: Strategies, Challenges, and Best Practices | OpenLogic, 5월 17, 2026에 액세스, <https://www.openlogic.com/blog/planning-spring-boot-4-migration>
22. Deploying a Java Project to Railway: A Step-by-Step Guide | by Sonu Jha - Medium, 5월 17, 2026에 액세스, <https://medium.com/@sonu-jha/deploying-a-java-project-to-railway-a-step-by-step-guide-e689340bd897>
23. Using Environment Variables in Spring Boot's Properties Files | Baeldung, 5월 17, 2026에 액세스, <https://www.baeldung.com/spring-boot-properties-env-variables>
24. Not able to connect my Spring Boot application to MySQL on Railway.app - Reddit, 5월 17, 2026에 액세스, https://www.reddit.com/r/SpringBoot/comments/1fgpqir/not_able_to_connect_my_spring_boot_application_to/
25. Spring boot environment Variables - Railway Central Station, 5월 17, 2026에 액세스, <https://station.railway.com/questions/spring-boot-environment-variables-d0c0bad0>
26. Vercel Rewrite: SPA Fallback except for API endpoints #5448 - GitHub, 5월 17, 2026에 액세스, <https://github.com/vercel/vercel/discussions/5448>
27. Nixpacks: Getting Started, 5월 17, 2026에 액세스, <https://nixpacks.com/>
28. java release version jdk 21 not supported - Central Station - Railway, 5월 17, 2026에 액세스, <https://station.railway.com/questions/java-release-version-jdk-21-not-supporte-3bb0eb32>
29. Deploy Your Spring Boot Application To Railway | by Mehedi hasan - Medium, 5월 17, 2026에 액세스, <https://meheedihasaan.medium.com/deploy-your-spring-boot-applications-to-railway-app-a2008b6d968c>